

Low-Rank Few-Shot Adaptation of Vision-Language Models

Maxime Zanella*
UCLouvain UMon

Ismail Ben Ayed
ÉTS Montréal

code: <https://github.com/MaxZanella/CLIP-LoRA>

Abstract

Recent progress in the few-shot adaptation of Vision-Language Models (VLMs) has further pushed their generalization capabilities, at the expense of just a few labeled samples within the target downstream task. However, this promising, already quite abundant few-shot literature has focused principally on prompt learning and, to a lesser extent, on adapters, overlooking the recent advances in Parameter-Efficient Fine-Tuning (PEFT). Furthermore, existing few-shot learning methods for VLMs often rely on heavy training procedures and/or carefully chosen, task-specific hyper-parameters, which might impede their applicability. In response, we introduce Low-Rank Adaptation (LoRA) in few-shot learning for VLMs, and show its potential on 11 datasets, in comparison to current state-of-the-art prompt- and adapter-based approaches. Surprisingly, our simple CLIP-LoRA method exhibits substantial improvements, while reducing the training times and keeping the same hyper-parameters in all the target tasks, i.e., across all the datasets and numbers of shots. Certainly, our surprising results do not dismiss the potential of prompt-learning and adapter-based research. However, we believe that our strong baseline could be used to evaluate progress in these emergent subjects in few-shot VLMs.

1. Introduction

Vision-Language Models (VLMs), such as CLIP [42], have emerged as powerful tools for learning cross-modal representations [23, 31, 42, 54, 58, 59]. Pre-trained on extensive collections of image-text pairs with a contrastive objective [42], VLMs learn to align these two modalities, enabling zero-shot prediction by matching the visual embeddings of the images and text descriptions (prompts) representing the target tasks. This joint representation

space of visual and textual features has also opened up new possibilities in the *Pre-training*, *Fine-tuning*, *Prediction* paradigm, through adaptation with very limited amounts of task-specific, labeled data [61, 63, 64], i.e., *few-shot adaptation*. Nonetheless, their efficacy often relies on the use of transformer-based architectures [49], where larger variants significantly outperform smaller ones. For instance, in CLIP, the ViT-L/14 model significantly surpasses the ViT-B/16 version by over 6% in accuracy on ImageNet [9]—a disparity that persists even following few-shot adaptation of the models. This underscores the need for efficient vision-language fine-tuning methods that are scalable to large models. The recent and emergent few-shot vision-language literature, although quite abundant already, has so far overlooked the computational overhead and memory footprint of fine-tuning strategies. This is the case of both the so-called *adapters*, which equip the models with additional trainable parameters [61], and popular *prompt-learning* methods [63], which fine-tune the input text prompts. This can increase the computational demand and the size of these already substantial models.

These questions surrounding the fine-tuning stage have triggered wide interest in NLP, where the increase in the sizes of foundational models is going at a fast pace, with some models boasting over 176 billion parameters [50], and even up to 540 billion [7]. To address these challenges, *Parameter Efficient Fine-Tuning (PEFT)* methods, which attempt to fine-tune only small amounts of parameters (in comparison to the original large models), have gained substantial attention [33]. Popular PEFT methods include selecting a subset of the existing parameters [56], adding small trainable modules called adapters [6, 20, 25, 34, 44], or adding trainable (prompt) tokens [24, 29, 32]. Recently, a novel approach consisting of solely fine-tuning low-rank matrices called Low-Rank Adaptation (LoRA) has appeared as a promising and practical method [21]. PEFT approaches, such as LoRA, have democratized the fine-tuning of large-language models, enabling even the management of billion-parameter models on a single GPU [11]. Far from merely enhancing computational

*Corresponding author: maxime.zanella@uclouvain.be

This work was partly supported by the Walloon Region (Service Public de Wallonie Recherche, Belgium) under grant n°2010235 (ARIAC by DigitalWallonia4.ai).

efficiency, empirical evidence has shown that, in the large-scale fine-tuning setting, LoRA could match or even exceed the performance of updating all model’s parameters [21]. *Although very promising/popular, and quite surprisingly, this fast-growing PEFT literature [11, 21, 27, 48, 60, 65] has found little echo in few-shot vision-language, where the dominant approaches have mainly focused on prompt tuning [3, 5, 53, 62–64] or adapters [14, 55, 61].*

The original CLIP paper demonstrated that better textual descriptions could greatly impact the zero-shot prediction [42]. This observation has been a strong motivation for the emergence of prompt tuning [29], a strategy that has been widely adopted within the vision-language community, following the seminal work of CoOp [63]. Indeed, CoOp popularized prompt tuning in the setting of few-shot VLMs. This has triggered a quite substantial recent literature focusing on improving prompt-learning performances for VLMs, in both the few-shot [3, 5, 10, 36, 53, 62–64] and unsupervised settings [13, 22, 38, 57]. While prompt learning methods improve the zero-shot performances, they incur heavy computational load for fine-tuning and might be hard to optimize, since every gradient update of the input requires back-propagating through the entire model; see the training times in Table 1.

Alongside this expanding prompt-tuning literature, there has been a few attempts to propose alternative approaches for few-shot VLMs, generally relying on adapters [14, 55, 61]. However, the performances of such adapters depend strongly on a set of hyper-parameters (such as the learning rate, number of epochs, or parameters controlling the blending of image and text embeddings) [45], which have to be found specifically for each target dataset. This is done via intensive searches over validation sets, requiring additional labeled samples and incurring computational overhead, which reduces their portability to new tasks.

Contributions. In this work, we investigate the deployment of Low-Rank Adaptation (LoRA) in the context of few-shot VLMs, an emergent, already quite abundant literature dominated by prompt-learning and adapter-based strategies. We thoroughly examine different design choices for deploying LoRA in this context, namely, the choices of the encoders (vision, language or both), of the specific weight matrices to adapt, and of the rank of the matrices. We conduct comprehensive empirical ablations and comparisons, over 11 datasets, emphasizing the best design choices for our baseline and juxtaposing it to the existing state-of-the-art prompt- and adapter-based methods. Surprisingly, our LoRA baseline beats the state-of-the-art in few-shot VLMs by important margins, while reducing the computational overhead. Furthermore, it relaxes the need for intensive searches of the hyper-parameters over

dataset-specific validation sets, maintaining a consistent hyper-parameter configuration across all the target tasks. While our surprising results do not invalidate the promise of prompt-learning and adapter-based strategies, we believe this strong baseline could be used to evaluate progress in these emergent subjects in few-shot VLMs.

2. Related work

Parameter-Efficient Fine-Tuning (PEFT). PEFT seeks to reduce the high expense of fine-tuning large-scale models by concentrating on (re-)training a relatively small number of parameters. These techniques can be categorized into four groups, primarily distinguished by the choice of parameters to train [33]. This often results in a trade-off among memory footprint, computational overhead, and performance. A summary is depicted in Figure 1.

The most straightforward way to avoid full fine-tuning is through *selective* methods, which focus on a subset of the existing model weights. Among these, BitFit [56] fine-tunes only the biases of both the attention and MLP layers in the transformer blocks, while other approaches prune the model to create a task-specific subset of parameters [15, 19].

Secondly, adapters integrate additional trainable modules into the original frozen architecture [6, 20, 25, 34, 44]; for example, by shifting and scaling deep features [34]. They also demonstrate their versatility and effectiveness in various tasks implying vision and language [47]. Nonetheless, the primary drawback of using adapters is the additional number of parameters after adaptation, which can lead to higher inference latency, even though some recent works aim at mitigating this issue [41].

Thirdly, there is prompt tuning or token-based tuning [24, 29, 32], which involves adding learnable tokens either to the input or at intermediate sequences. This strategy has been particularly popular in vision-language for few-shot and zero-shot learning, replacing hard-to-design template prompts with learnable soft ones [38, 63]. Initially applied to textual prompts, recent works have extended this technique to train visual tokens within transformer-based architectures [24]. This research direction has begun to spark interest in the few-shot vision-language field [26].

Finally, Low-Rank Adaptation (LoRA) [21] adds low-rank matrices to explicitly represent weight changes while keeping the original parameters frozen. These explicit changes can then be merged with original weights prior inference, inducing no additional inference latency in comparison to the vanilla model. LoRA operates on the hypothesis that updates required for the fine-tuning process exhibit a low “intrinsic rank” [1, 30], a property we can directly control with the rank of each weight change matrix. In this respect, LoRA can be viewed as an adapter approach, yet it offers the advantages of selective methods by providing a direct aggregation of its module, eliminating the need for

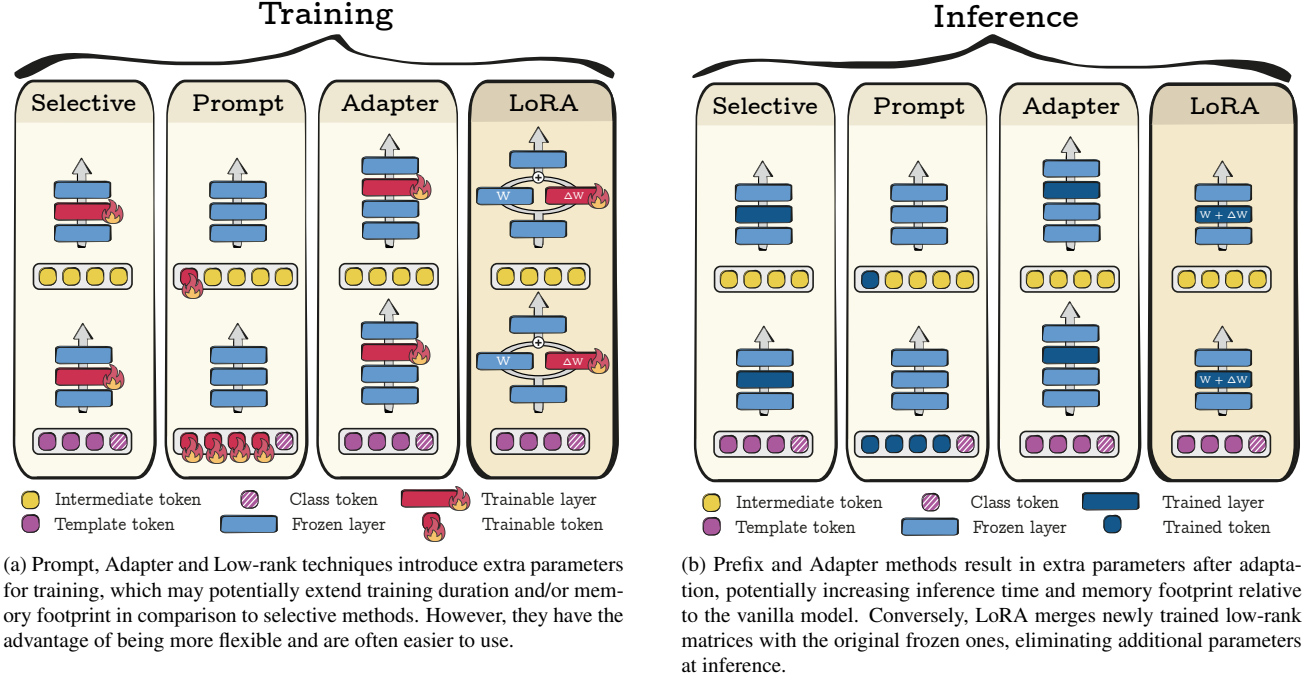


Figure 1. Different categories of Parameter-Efficient Fine-Tuning (PEFT) methods during (a) training, and (b) inference.

extra parameters at the inference stage. Several versions of the original LoRA have since appeared, some focusing on making the rank adaptive for each matrix [48, 60], others pushing its performance [4, 27, 65] or reducing its memory footprint through quantization [11, 43].

Few-shot learning in Vision-Language. Large scale VLMs have shown excellent results in several vision tasks [59]. This success has created interest in developing adaptation techniques that capitalize on their general knowledge [51]. Among these, prompt tuning [29] has emerged as the primary method for adapting VLMs with few labeled data [3, 5, 10, 36, 53, 62–64]. CoOp [63] optimizes learnable common continuous tokens attached to the class names, described as a context optimization. CoCoOp [62] trains a neural network to generate instance-conditioned tokens based on the image. Further efforts like ProGrad [64] and KgCoOp [53], among others [3], guide prompts towards predefined handcrafted ones, for example, thanks to gradients projection [64] with the idea of preserving initial knowledge during learning.

Among prompt-learning works, PLOT [5] is one of the first to adapt jointly the text and image modalities. They propose to align learned prompts with finer-grained visual features through an optimal transport formulation. Note that this cross-modal adaptation is also a key factor in our approach, as discussed in Section 6. Following a similar trajectory, MaPLe [26] introduces intermediate learnable to-

kens within both the vision and text encoders, while making them interdependent at each level. They further demonstrate that adapting both modality branches allows for more flexibility in the downstream tasks.

Adapter-based methods offer an alternative strategy and are increasingly studied in vision-language [14, 55, 61]. CLIP-Adapter [14] learns visual adapters to combine adapted and original features. A few other methods propose to leverage the knowledge of these models while only accessing their final embedding state. Examples include parameter-free plug-in attention for the zero-shot scenario [16], or Tip-Adapter(-F) [61] using a cache model in few-shot learning. In a similar vein, TaskRes [55] keeps the original text weights frozen and introduces task residual tuning to learn task-specific adapters built on the initial knowledge.

3. Few-shot fine-tuning for VLMs

This section provides a broad overview of recent few-shot fine-tuning methods designed for VLMs, summarized in Figure 1. First, let us introduce a few basic notations and definitions.

When dealing with a classification task based on a vision-language model, and given a set of K candidate classes, one creates textual descriptions, the so-called prompts [35], each corresponding to a class, e.g., $\mathbf{c}_k$ is the tokenized version of a photo of a [kth class name], $k = 1, \dots, K$. Let $\mathbf{t}_k = \theta_t(\mathbf{c}_k)$ de-

Table 1. Training time on 16-shots ImageNet task. Experiments were conducted on a single A100 80Gb with the original code provided by the authors. For PLOT++ the time reported includes the 2 training stages.

Method	Training time
CoOp (16)	2h
PLOT++	15h30
ProGrad	3h20
CLIP-LoRA	50 min.

notes the corresponding normalized (unit-hypersphere) textual embedding representation, with θ_t representing the parameters of the language encoder. Similarly, each image $\mathbf{x}_i$, $i = 1, \dots, N$, is projected onto a normalized embedding space of the same dimension, using the visual encoder θ_v : $\mathbf{f}_i = \theta_v(\mathbf{x}_i)$.

The zero-shot prediction is the simplest form of adapting VLMs to a downstream task, which follows their pre-training procedure [42]. Pairing each text embedding $\mathbf{t}_k$ with $\mathbf{f}_i$, the visual embedding of test image $\mathbf{x}_i$, one could measure their cosine similarity, yielding a prediction logit:

$$l_{i,k} = \mathbf{f}_i^\top \mathbf{t}_k. \quad (1)$$

This also yields a probabilistic prediction, in the form of a posterior softmax probability of class k given test input $\mathbf{x}_i$:

$$p_{i,k} = \frac{\exp(l_{i,k}/\tau)}{\sum_j^K \exp(l_{i,j}/\tau)} \quad (2)$$

where τ is a softmax-temperature parameter¹. Hence, zero-shot classification of image $\mathbf{x}_i$ is done by finding the class with the highest posterior probability: $\hat{k} = \operatorname{argmax}_k p_{i,k}$.

In the few-shot setting, and to further adapt these models, we assume that we have access to N/K labeled samples for each target class, the so-called *support* set. N denotes the total number of support samples, and N/K (the number of shots per class) is typically small (less than 16). Let y_{ik} denotes the one-hot encoded label for a labeled support image $\mathbf{x}_i$, i.e., $y_{ik} = 1$ if k is the class of image $\mathbf{x}_i$ and 0 otherwise. Then, we minimize the cross-entropy (CE) loss:

$$-\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \ln p_{i,k} \quad (3)$$

This is done either (i) by fine-tuning the input prompts, $\mathbf{c}_k$, $k = 1, \dots, K$, as in prompt-tuning methods following on from the pioneering work of CoOp [5, 26, 53, 62–64]; or (ii) by updating a set of additional parameters, as in adapters

¹Note that each CLIP version comes with a temperature scaling τ , which is optimized along with the learnable parameters during pre-training.

[14, 55, 61]. Note that other methods propose to tune additional intermediate tokens [26], which we include under the category “prompt tuning” for a more general terminology. We will now detail the two current strategies used in VLMs: Prompt tuning (P) and Adapters (A).

Prompt tuning (P). The way prompting is performed in VLMs could significantly impact the ensuing performances [42]. To address this issue, soft prompt tuning [29, 32] optimizes text-input tokens, which could be extended to intermediate layers [32]. Similarly, if a transformer-based [49] architecture is used, these learnable tokens can be inserted in vision models [24]. In the context of few-shot VLMs, the authors of [63] introduced context optimization (CoOp), which constructs text input $\mathbf{c}_k$ as continuous trainable vectors:

$$\mathbf{c}_k = (\mathbf{v}_k^1, \dots, \mathbf{v}_k^M, [\text{class}_k]) \quad (4)$$

Where M denotes a hyper-parameter, $(\mathbf{v}_k^l)_{1 \leq l \leq M}$ are trainable text tokens, and $[\text{class}_k]$ is a fixed token. The latter is the word embedding vector of the name of the k^{th} class. These trainable vectors are updated as task-specific text prompts by using the standard supervised CE classification loss in Eq. (3), along with the few-shot labeled samples. Prompt tuning has a clear advantage over adapter-based methods: They remove the need for heuristically choosing the text prompts [63], which are specifically engineered for each task, and whose choice might affect the performances significantly. While prompt-tuning methods improves significantly the performances of classification, they incur heavy computational load for fine-tuning and might be hard to optimize, since every gradient update of the text input requires back-propagating through the entire model (see the training times reported in Table 1).

Adapters (A). Instead of updating the text prompts, another class of methods, called adapters, augment the pre-trained model with extra parameters while keeping the existing ones frozen [20]. This provides an efficient way to control the number of trainable parameters. The idea has been recently explored in the few-shot vision-language setting [14, 55, 61]. In this setting, adapters could be viewed as feature transformations, via some multi-layer modules, appended to the encoder’s bottleneck. This enables to learn transformations blending image and text features, with logits taking the following form:

$$l_{i,k} = \theta_a(\mathbf{f}_i, \mathbf{t}_k) \quad (5)$$

where θ_a denotes the additional trainable parameters of the adapter. These are fine-tuned by minimizing the CE loss in (3), using the labeled support set but now with logits $l_{i,k}$ transformed by (5). For example, CLIP-Adapter [14]

added a multi-layered perceptron to modify the features. Tip-Adapter [61] added a module, which evaluates class scores via some pairwise similarities between the features of the labeled samples, and integrates these scores with the embeddings of the text prompts. This class of methods reduce the computational load in comparison with prompt-tuning techniques. However, as pointed out recently in the experiments in [45], their performances seem to depend strongly on some key hyper-parameters that have to be adjusted specifically for each downstream task. This is done via intensive searches over validation sets, requiring additional labeled samples [61] and incurring computational overhead, which reduces their portability to new tasks.

4. CLIP-LoRA

Low-Rank Adaptation (LoRA) [21] models the incremental update of the pre-trained weights as the product of two small matrices, $\mathbf{A}$ and $\mathbf{B}$, based on the idea of “intrinsic rank” of a downstream task. For an input $\mathbf{x}$, a hidden state $\mathbf{h}$, and a weight matrix $\mathbf{W} \in \mathbb{R}^{d_1 \times d_2}$, the modified forward pass, following the application of a LoRA module, is:

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \gamma\Delta\mathbf{W}\mathbf{x} = \mathbf{W}\mathbf{x} + \gamma\mathbf{B}\mathbf{A}\mathbf{x} \quad (6)$$

where $\mathbf{A} \in \mathbb{R}^{r \times d_2}$, $\mathbf{B} \in \mathbb{R}^{d_1 \times r}$, $\Delta\mathbf{W} \in \mathbb{R}^{d_1 \times d_2}$ of rank r , with r typically $\ll \{d_1, d_2\}$, and γ a scaling factor. Values in $\mathbf{A}$ are randomly initialized via Kaiming initialization while $\mathbf{B}$ is filled with zeros. This implies that there is no incremental update before training, and therefore, the output remains unchanged.

In the original LoRA paper, the low-rank matrices are applied on the attention matrices of transformer-based architectures [49]. They typically consist of L stacked blocks, each containing a multi-head attention (MHA) module:

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{x}\mathbf{W}_{q_i}(\mathbf{x}\mathbf{W}_{k_i})^T}{\sqrt{d}}\right)(\mathbf{x}\mathbf{W}_{v_i})$$

$$\text{MHA}(\mathbf{x}) = \text{concat}(\text{head}_1, \dots, \text{head}_H)\mathbf{W}_o$$

where d is a scaling factor and $\mathbf{W}_{K_i}$, $\mathbf{W}_{Q_i}$, $\mathbf{W}_{V_i}$, $\mathbf{W}_o$ are weight matrices, corresponding respectively to the key, query, value and output matrices. Note that other works extend this approach to the feed-forward module’s weight matrices [17].

LoRA for VLMs. A straightforward way to apply LoRA in vision-language is to apply it to all the matrices of the vision and text encoders. However, due to the relatively small supervision inherent to the few-shot setting, we only apply low-rank matrices on the query, key and value matrices with $r = 2$. We regularize the input of the LoRA module by a dropout layer with $p = 0.25$ [21]. The number of iterations is set equal to 500 times N/K (the number

of labeled samples per class). We used a learning rate of $2 * 10^{-4}$, with a cosine scheduler and a batch size of 32, so that all training could be performed on a single GPU of 24 Gb. *These hyper-parameters are kept fixed across all the experiments.* The input prompt is simply set to a `photo of a [kth class name]`, $k = 1, \dots, K$, for every dataset, to emphasize the applicability of CLIP-LoRA without resorting to complex initial manual prompting. Note that the LoRA modules are positioned at every levels of both encoders. The impact of the location of the LoRA modules is studied in Section 6, putting in evidence that adapting both modalities can be necessary for certain tasks.

5. Few-shot learning

We follow the setting of previous work [63]. We consider 10 datasets for fine-grained classification of scenes (SUN397 [52]), aircraft types (Aircraft [37]), satellite imagery (EuroSAT [18]), automobiles (Stanford-Cars [28]), food items (Food101 [2]), pet breeds (OxfordPets [40]), flowers (Flower102 [39]), general objects (Caltech101 [12]), textures (DTD [8]) and human actions (UCF101 [46]) as well as ImageNet [9]. These datasets offer a thorough benchmarking framework for evaluating few-shot visual classification tasks.

Comparative methods. We compare CLIP-LoRA to several prompt-based methods: CoOp [63] (4) with 4 learnable tokens, CoOp [63] (16) with 16 learnable tokens, Co-CoOp [62], PLOT++ [5] which is an adaption of the original PLOT proposed by the same authors specifically designed for transformer architectures, KgCoOp [53], MaPLE [26] for which we follow the training procedure of their “base-to-new” setting, ProGrad [64] with 16 tokens. We also report adapter-based methods: Tip-Adapter-F [61] for which we reduce the validation set to a reasonable size of $\min(n_shots, 4)$, TaskRes [55] for which we only report the not enhanced base performance due to its unavailability for all datasets/shots/backbones studied in this paper. Despite some questionable arbitrary choices as discussed in [45], we keep their specific hyper-parameters [45] while CLIP-LoRA uses the same hyper-parameters for every tasks.

CLIP-LoRA outperforms, on average, adapter- and prompt-based few-shot methods. The strongest adapter-based method in Table 2 is Tip-Adapter-F, which is not competitive with CLIP-LoRA despite relying heavily on arbitrary hyper-parameters for each dataset (namely the starting value of their α, β as well as the search range during validation). We can conclude the same for TaskRes, which also relies on arbitrary choices for a given dataset, i.e., a specific learning rate for ImageNet and a specific scaling factor for the Flowers dataset. Regarding prompt-based approaches,

Table 2. Detailed results for 11 datasets with the ViT-B/16 as visual backbone. Top-1 accuracy averaged over 3 random seeds is reported. Highest value is highlighted in **bold**, and the second highest is underlined.

Shots	Method	ImageNet	SUN	Aircraft	EuroSAT	Cars	Food	Pets	Flowers	Caltech	DTD	UCF	Average
0	CLIP (ICML '21)	66.7	62.6	24.7	47.5	65.3	86.1	89.1	71.4	92.9	43.6	66.7	65.1
1	CoOp (4) (IJCV '22)	68.0	67.3	26.2	50.9	67.1	82.6	90.3	72.7	93.2	50.1	70.7	67.2
	CoOp (16) (IJCV '22)	65.7	67.0	20.8	56.4	67.5	84.3	90.2	78.3	92.5	50.1	71.2	67.6
	CoCoOp (CVPR '22)	69.4	68.7	28.1	55.4	67.6	84.9	91.9	73.4	94.1	52.6	70.4	68.8
	TIP-Adapter-F (ECCV '22)	69.4	67.2	28.8	<u>67.8</u>	67.1	85.8	90.6	83.8	94.0	51.6	73.4	<u>70.9</u>
	CLIP-Adapter (IJCV '23)	67.9	65.4	25.2	49.3	65.7	86.1	89.0	71.3	92.0	44.2	66.9	65.7
	PLOT++ (ICLR '23)	66.5	66.8	28.6	65.4	68.8	<u>86.2</u>	91.9	80.5	94.3	54.6	<u>74.3</u>	70.7
	KgCoOp (CVPR '23)	68.9	68.4	26.8	61.9	66.7	86.4	<u>92.1</u>	74.7	<u>94.2</u>	52.7	72.8	69.6
	TaskRes (CVPR '23)	69.6	68.1	31.3	65.4	<u>68.8</u>	84.6	90.2	81.7	93.6	53.8	71.7	70.8
	MaPLe (CVPR '23)	<u>69.7</u>	<u>69.3</u>	28.1	29.1	67.6	85.4	91.4	74.9	93.6	50.0	71.1	66.4
	ProGrad (ICCV '23)	67.0	67.0	28.8	57.0	68.2	84.9	91.4	80.9	93.5	52.8	73.3	69.5
	CLIP-LoRA (Ours)	70.4	70.4	<u>30.2</u>	72.3	70.1	84.3	92.3	<u>83.2</u>	93.7	<u>54.3</u>	76.3	72.5
4	CoOp (4) (IJCV '22)	69.7	70.6	29.7	65.8	73.4	83.5	92.3	86.6	94.5	58.5	78.1	73.0
	CoOp (16) (IJCV '22)	68.8	69.7	30.9	69.7	74.4	84.5	92.5	92.2	94.5	59.5	77.6	74.0
	CoCoOp (CVPR '22)	70.6	70.4	30.6	61.7	69.5	86.3	<u>92.7</u>	81.5	94.8	55.7	75.3	71.7
	TIP-Adapter-F (ECCV '22)	70.7	70.8	<u>35.7</u>	76.8	74.1	86.5	91.9	92.1	94.8	59.8	78.1	75.6
	CLIP-Adapter (IJCV '23)	68.6	68.0	27.9	51.2	67.5	86.5	90.8	73.1	94.0	46.1	70.6	67.7
	PLOT++ (ICLR '23)	70.4	71.7	35.3	<u>83.2</u>	<u>76.3</u>	86.5	92.6	<u>92.9</u>	<u>95.1</u>	<u>62.4</u>	<u>79.8</u>	<u>76.9</u>
	KgCoOp (CVPR '23)	69.9	71.5	32.2	71.8	69.5	86.9	92.6	87.0	95.0	58.7	77.6	73.9
	TaskRes (CVPR '23)	<u>71.0</u>	<u>72.7</u>	33.4	74.2	76.0	86.0	91.9	85.0	95.0	60.1	76.2	74.7
	MaPLe (CVPR '23)	70.6	71.4	30.1	69.9	70.1	<u>86.7</u>	93.3	84.9	95.0	59.0	77.1	73.5
	ProGrad (ICCV '23)	70.2	71.7	34.1	69.6	75.0	85.4	92.1	91.1	94.4	59.7	77.9	74.7
	CLIP-LoRA (Ours)	71.4	72.8	37.9	84.9	77.4	82.7	91.0	93.7	95.2	63.8	81.1	77.4
16	CoOp (4) (IJCV '22)	71.5	74.6	40.1	83.5	79.1	85.1	92.4	96.4	95.5	69.2	81.9	79.0
	CoOp (16) (IJCV '22)	71.9	74.9	43.2	85.0	82.9	84.2	92.0	96.8	95.8	69.7	83.1	80.0
	CoCoOp (CVPR '22)	71.1	72.6	33.3	73.6	72.3	87.4	<u>93.4</u>	89.1	95.1	63.7	77.2	75.4
	TIP-Adapter-F (ECCV '22)	<u>73.4</u>	<u>76.0</u>	44.6	85.9	82.3	86.8	92.6	96.2	95.7	70.8	83.9	80.7
	CLIP-Adapter (IJCV '23)	69.8	74.2	34.2	71.4	74.0	87.1	92.3	92.9	94.9	59.4	80.2	75.5
	PLOT++ (ICLR '23)	72.6	<u>76.0</u>	<u>46.7</u>	<u>92.0</u>	<u>84.6</u>	87.1	93.6	<u>97.6</u>	<u>96.0</u>	71.4	<u>85.3</u>	<u>82.1</u>
	KgCoOp (CVPR '23)	70.4	73.3	36.5	76.2	74.8	<u>87.2</u>	93.2	93.4	95.2	68.7	81.7	77.3
	TaskRes (CVPR '23)	73.0	76.1	44.9	82.7	83.5	86.9	92.4	97.5	95.8	<u>71.5</u>	84.0	80.8
	MaPLe (CVPR '23)	71.9	74.5	36.8	87.5	74.3	87.4	93.2	94.2	95.4	68.4	81.4	78.6
	ProGrad (ICCV '23)	72.1	75.1	43.0	83.6	82.9	85.8	92.8	96.6	95.9	68.8	82.7	79.9
	CLIP-LoRA (Ours)	73.6	76.1	54.7	92.1	86.3	84.2	92.4	98.0	96.4	72.0	86.7	83.0

Table 2 shows that CoOp and ProGrad are outperformed by a large margin. The strongest competitor is, without a doubt, PLOT++. PLOT++ necessitates a two-stage training (each of 50 epochs for ImageNet) as well as several dataset-specific textual templates for their optimal transport formulation, reducing its portability to other downstream tasks. Overall, CLIP-LoRA performs better, especially on ImageNet, UCF101 and Aircraft, while being more practical. However, it underperforms on two datasets, Food101 and OxfordPet, where few-shot learning offers minimal improvement. This may be attributed to the lack of regularization, considering we use straightforward cross-entropy loss. We observe a similar trend with CoOp, whereas approaches that incorporate explicit regularization, such as ProGrad, do not exhibit this issue. Note that more detailed results, including for 2 and 8 shots, are available in the Appendix.

CLIP-LoRA performances are consistent across various vision encoders. As depicted in Figure 2, CLIP-LoRA surpasses, on average, the other few-shot methods with both the ViT-B/32 architecture and the larger ViT-L/14. This further supports the versatility of our approach. Detailed results for the three backbones are available in the Appendix.

CLIP-LoRA is computationally and memory efficient. Table 1 compares the training time of the leading prompt-learning methods; CLIP-LoRA achieves better performance with shorter training. Moreover, the best performing adapter method, namely Tip-Adapter-F, depends on a large cache model that stores embeddings for all instances across every class. In contrast, LoRA merges its adapted matrices at the inference stage, thereby eliminating the need for extra memory beyond what is required by the original model.

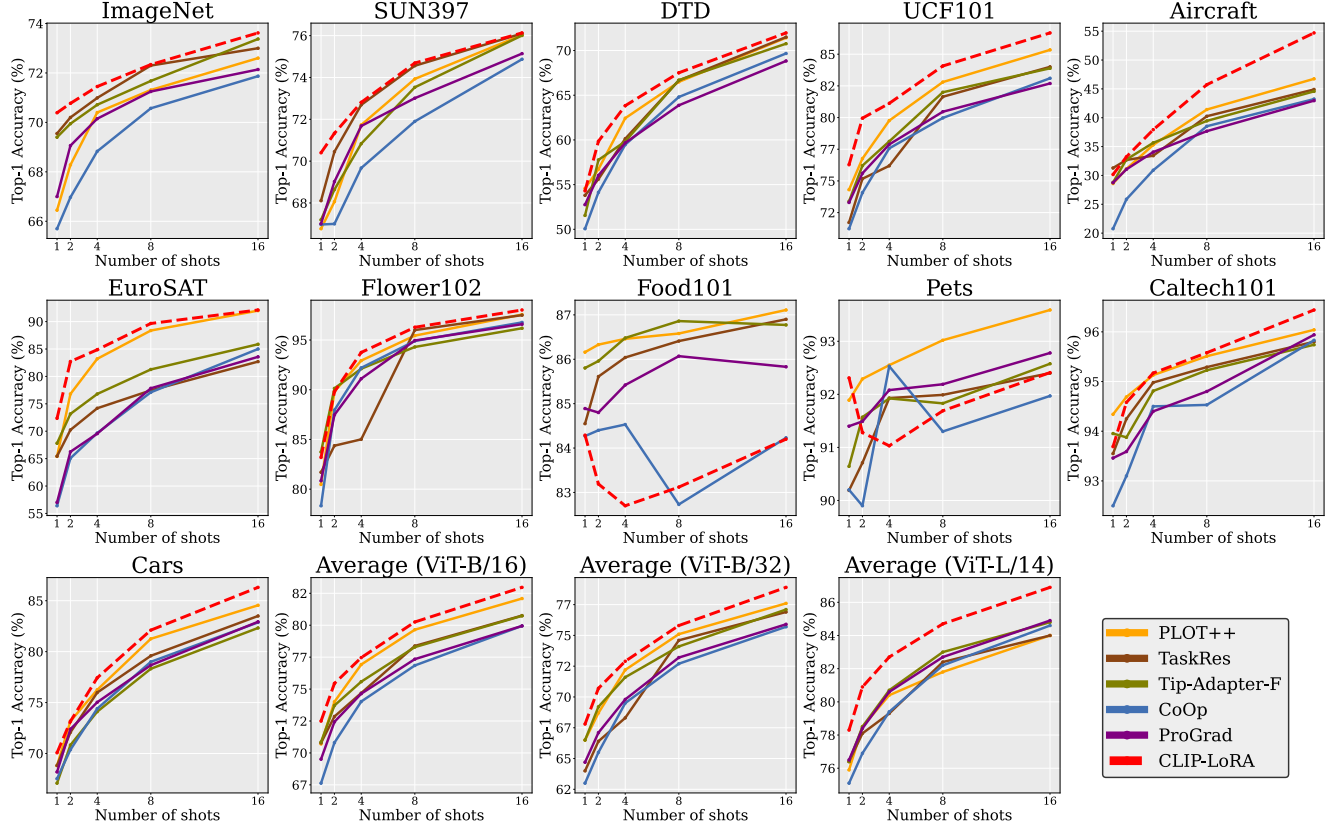


Figure 2. Detailed few-shot learning results on the 10 fine-grained datasets and ImageNet with the ViT-B/16 visual backbone. Average performance for the ViT-B/16, ViT-B/32 and ViT-L/14 on the same 11 datasets is reported in the last three plots, respectively.

6. How to apply LoRA for VLMs?

In this section, we delve into the utilization of LoRA modules, identifying three principal design considerations: (1) the choice between tuning the vision encoder, the text encoder, or both, including the specific layers to adjust; (2) the selection of attention matrices for tuning; and (3) the determination of the appropriate rank for these matrices. We explore these aspects across three datasets: ImageNet, Stanford Cars, and EuroSAT. ImageNet was selected for its broad diversity, while the latter two were chosen for their distinctive behaviors. Results are depicted in Figure 3 for seven different groups of adapted attention matrices and increasing rank value.

Adapting both encoders leads to the best results on average. With the exception of EuroSAT, where adapting solely the vision encoder shows marginally better stability, tuning both encoders concurrently is the most effective strategy, leading to significant enhancements. This aligns with recent approaches that incorporate additional vision tokens [5, 26] to augment performance beyond what is achievable with text-only prompt tuning, as seen in CoOp [63].

Tuning more attention matrices can lead to better results but... Among the four attention matrices studied, adapting value or output matrices ($\mathbf{W}_v$ and $\mathbf{W}_o$) appears to be the best strategy, showing quite consistent differences in performance. Moreover, as discussed in the original LoRA paper and subsequent works [21, 60], adapting a larger number of weight matrices can lead to better results. However, it can also decrease performance, as demonstrated on ImageNet and StanfordCars with high rank. This is in line with recent methods that aim to dynamically adjust the rank of the matrices [48, 60].

Choosing the location of LoRA modules requires careful consideration. The impact of LoRA module placement—whether on the lower half (bottom), or the upper half (up)—is illustrated in the bar plots of Figure 3 with varying performance and no clear winner. We found it more effective to add LoRA modules across all layers. In comparison, in the context of LLMs, AdaLoRA [60] suggests that allocating a larger rank to the middle and last layers rather than the first ones yields better results. Similar strategies applied for VLMs could reveal promising avenues for future research.

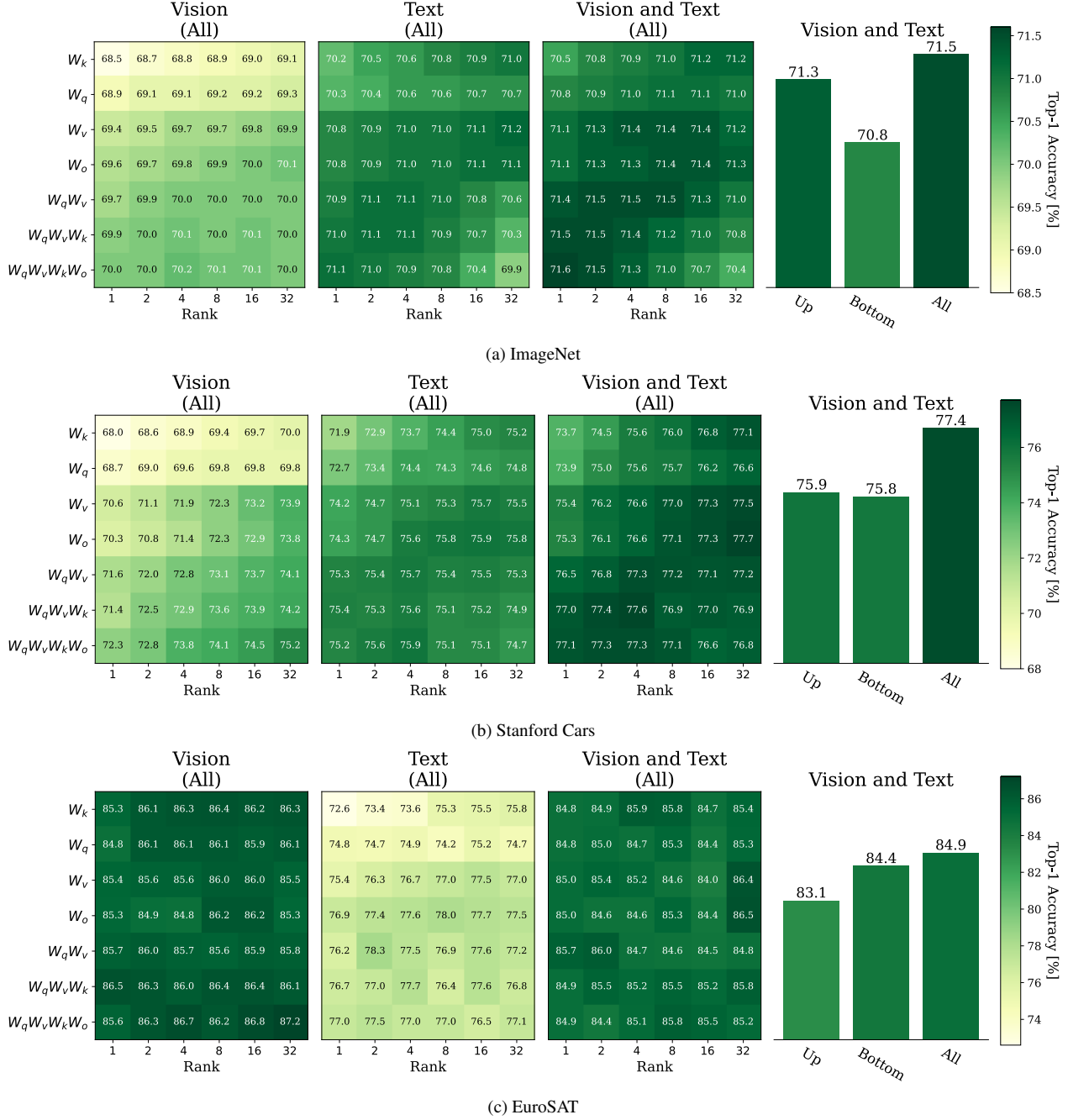


Figure 3. Top-1 accuracy with 4-shots for different matrices of the attention bloc and increasing rank, when the low-rank matrices are positioned at every level of the encoders (All). The fourth bar plot study the impact of positioning the low-rank matrices only on the half last levels (Up), the first half levels (Bottom), or at every level (All). Reported top-1 accuracy is averaged over 3 random seeds.

7. Conclusion

We established a strong baseline by consistently outperforming prompt- and adapter-based methods in few-shot adaptation of Vision-Language Models (VLMs) using fixed hyper-parameters. We hope our work inspires future efforts to design methods that either uphold this simplicity and effi-

ciency with fixed hyper-parameters or offer clear guidelines for adaptable hyper-parameter settings. Additionally, we demonstrated that selecting the matrices to adapt and determining the corresponding rank to maximize performance using LoRA modules is not trivial. We believe these aspects of our work suggest a promising area for future research.

References

- [1] Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7319–7328, 2021. [2](#)
- [2] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101—mining discriminative components with random forests. In *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part VI 13*, pages 446–461. Springer, 2014. [5](#)
- [3] Adrian Bulat and Georgios Tzimiropoulos. Lasp: Text-to-text optimization for language-aware soft prompting of vision & language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23232–23241, 2023. [2, 3](#)
- [4] Arnav Chavan, Zhuang Liu, Deepak Gupta, Eric Xing, and Zhiqiang Shen. One-for-all: Generalized lora for parameter-efficient fine-tuning. *arXiv preprint arXiv:2306.07967*, 2023. [3](#)
- [5] Guangyi Chen, Weiran Yao, Xiangchen Song, Xinyue Li, Yongming Rao, and Kun Zhang. Plot: Prompt learning with optimal transport for vision-language models. In *The Eleventh International Conference on Learning Representations*, 2022. [2, 3, 4, 5, 7](#)
- [6] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. Adaptformer: Adapting vision transformers for scalable visual recognition. *Advances in Neural Information Processing Systems*, 35:16664–16678, 2022. [1, 2](#)
- [7] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240): 1–113, 2023. [1](#)
- [8] Mircea Cimpoi, Subhansu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3606–3613, 2014. [5](#)
- [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. [1, 5](#)
- [10] Mohammad Mahdi Derakhshani, Enrique Sanchez, Adrian Bulat, Victor Guilherme Turrissi da Costa, Cees GM Snoek, Georgios Tzimiropoulos, and Brais Martinez. Variational prompt tuning improves generalization of vision-language models. *arXiv preprint arXiv:2210.02390*, 2022. [2, 3](#)
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024. [1, 2, 3](#)
- [12] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *2004 conference on computer vision and pattern recognition workshop*, pages 178–178. IEEE, 2004. [5](#)
- [13] Chun-Mei Feng, Kai Yu, Yong Liu, Salman Khan, and Wangmeng Zuo. Diverse data augmentation with diffusions for effective test-time prompt tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2704–2714, 2023. [2](#)
- [14] Peng Gao, Shijie Geng, Renrui Zhang, Teli Ma, Rongyao Fang, Yongfeng Zhang, Hongsheng Li, and Yu Qiao. Clip-adapter: Better vision-language models with feature adapters. *International Journal of Computer Vision*, pages 1–15, 2023. [2, 3, 4](#)
- [15] Demi Guo, Alexander M Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4884–4896, 2021. [2](#)
- [16] Ziyu Guo, Renrui Zhang, Longtian Qiu, Xianzheng Ma, Xupeng Miao, Xuming He, and Bin Cui. Calip: Zero-shot enhancement of clip with parameter-free attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 746–754, 2023. [3](#)
- [17] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. In *International Conference on Learning Representations*, 2021. [5](#)
- [18] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019. [5](#)
- [19] Connor Holmes, Minjia Zhang, Yuxiong He, and Bo Wu. Nxmttransformer: semi-structured sparsification for natural language understanding via admn. *Advances in neural information processing systems*, 34:1818–1830, 2021. [2](#)
- [20] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019. [1, 2, 4](#)
- [21] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2021. [1, 2, 5, 7](#)
- [22] Tony Huang, Jack Chu, and Fangyun Wei. Unsupervised prompt learning for vision-language models. *arXiv preprint arXiv:2204.03649*, 2022. [2](#)
- [23] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *International conference on machine learning*, pages 4904–4916. PMLR, 2021. [1](#)

- [24] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision*, pages 709–727. Springer, 2022. 1, 2, 4
- [25] Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers. In *Advances in Neural Information Processing Systems*, pages 1022–1035. Curran Associates, Inc., 2021. 1, 2
- [26] Muhammad Uzair Khattak, Hanoona Rasheed, Muhammad Maaz, Salman Khan, and Fahad Shahbaz Khan. Maple: Multi-modal prompt learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19113–19122, 2023. 2, 3, 4, 5, 7
- [27] Sanghyeon Kim, Hyunmo Yang, Younghyun Kim, Youngjoon Hong, and Eunbyung Park. Hydra: Multi-head low-rank adaptation for parameter efficient fine-tuning. *arXiv preprint arXiv:2309.06922*, 2023. 2, 3
- [28] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *Proceedings of the IEEE international conference on computer vision workshops*, pages 554–561, 2013. 5
- [29] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021. 1, 2, 3, 4
- [30] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations*, 2018. 2
- [31] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, et al. Grounded language-image pre-training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10965–10975, 2022. 1
- [32] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, 2021. 1, 2, 4
- [33] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023. 1, 2
- [34] Dongze Lian, Daquan Zhou, Jiashi Feng, and Xinchao Wang. Scaling & shifting your features: A new baseline for efficient model tuning. *Advances in Neural Information Processing Systems*, 35:109–123, 2022. 1, 2
- [35] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9): 1–35, 2023. 3
- [36] Yuning Lu, Jianzhuang Liu, Yonggang Zhang, Yajing Liu, and Xinmei Tian. Prompt distribution learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5206–5215, 2022. 2, 3
- [37] Subhansu Maji, Esa Rahtu, Juho Kannala, Matthew Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151*, 2013. 5
- [38] Shu Manli, Nie Weili, Huang De-An, Yu Zhiding, Goldstein Tom, Anandkumar Anima, and Xiao Chaowei. Test-time prompt tuning for zero-shot generalization in vision-language models. In *NeurIPS*, 2022. 2
- [39] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian conference on computer vision, graphics & image processing*, pages 722–729. IEEE, 2008. 5
- [40] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3498–3505. IEEE, 2012. 5
- [41] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 487–503, 2021. 2
- [42] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. 1, 2, 4
- [43] Hossein Rajabzadeh, Mojtaba Valipour, Tianshu Zhu, Marzieh Tahaei, Hyock Ju Kwon, Ali Ghodsi, Boxing Chen, and Mehdi Rezagholizadeh. Qdylora: Quantized dynamic low-rank adaptation for efficient large language model tuning. *arXiv preprint arXiv:2402.10462*, 2024. 3
- [44] Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *Advances in neural information processing systems*, 30, 2017. 1, 2
- [45] Julio Silva-Rodriguez, Sina Hajimiri, Ismail Ben Ayed, and Jose Dolz. A closer look at the few-shot adaptation of large vision-language models. *arXiv preprint arXiv:2312.12730*, 2023. 2, 5
- [46] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 5
- [47] Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. Vi-adapter: Parameter-efficient transfer learning for vision-and-language tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5227–5237, 2022. 2
- [48] Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan Kobayev, and Ali Ghodsi. Dylora: Parameter efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. *arXiv preprint arXiv:2210.07558*, 2022. 2, 3, 7
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 4, 5

- [50] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022. [1](#)
- [51] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, et al. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7959–7971, 2022. [3](#)
- [52] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE computer society conference on computer vision and pattern recognition*, pages 3485–3492. IEEE, 2010. [5](#)
- [53] Hantao Yao, Rui Zhang, and Changsheng Xu. Visual-language prompt tuning with knowledge-guided context optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6757–6767, 2023. [2](#), [3](#), [4](#), [5](#)
- [54] Lewei Yao, Runhui Huang, Lu Hou, Guansong Lu, Minzhe Niu, Hang Xu, Xiaodan Liang, Zhenguo Li, Xin Jiang, and Chunjing Xu. Filip: Fine-grained interactive language-image pre-training. *arXiv preprint arXiv:2111.07783*, 2021. [1](#)
- [55] Tao Yu, Zhihe Lu, Xin Jin, Zhibo Chen, and Xinchao Wang. Task residual for tuning vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10899–10909, 2023. [2](#), [3](#), [4](#), [5](#)
- [56] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–9, 2022. [1](#), [2](#)
- [57] Maxime Zanella and Ismail Ben Ayed. On the test-time zero-shot generalization of vision-language models: Do we really need prompt learning? *arXiv preprint arXiv:2405.02266*, 2024. [2](#)
- [58] Xiaohua Zhai, Xiao Wang, Basil Mustafa, Andreas Steiner, Daniel Keysers, Alexander Kolesnikov, and Lucas Beyer. Lit: Zero-shot transfer with locked-image text tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18123–18133, 2022. [1](#)
- [59] Jingyi Zhang, Jiaying Huang, Sheng Jin, and Shijian Lu. Vision-language models for vision tasks: A survey. *arXiv preprint arXiv:2304.00685*, 2023. [1](#), [3](#)
- [60] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations*, 2022. [2](#), [3](#), [7](#)
- [61] Renrui Zhang, Wei Zhang, Rongyao Fang, Peng Gao, Kunchang Li, Jifeng Dai, Yu Qiao, and Hongsheng Li. Tip-adapter: Training-free adaption of clip for few-shot classification. In *European Conference on Computer Vision*, pages 493–510. Springer, 2022. [1](#), [2](#), [3](#), [4](#), [5](#)
- [62] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Conditional prompt learning for vision-language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. [2](#), [3](#), [4](#), [5](#)
- [63] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision (IJCV)*, 2022. [1](#), [2](#), [3](#), [4](#), [5](#), [7](#)
- [64] Beier Zhu, Yulei Niu, Yucheng Han, Yue Wu, and Hanwang Zhang. Prompt-aligned gradient for prompt tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15659–15669, 2023. [1](#), [2](#), [3](#), [4](#), [5](#)
- [65] Bojia Zi, Xianbiao Qi, Lingzhi Wang, Jianan Wang, Kam-Fai Wong, and Lei Zhang. Delta-lora: Fine-tuning high-rank parameters with the delta of low-rank matrices. *arXiv preprint arXiv:2309.02411*, 2023. [2](#), [3](#)

Table 3. Detailed results for the 11 datasets with ViT-B/16 as backbone. Top-1 accuracy averaged over 3 random seeds is reported. Highest value is highlighted in **bold**, and the second highest is underlined.

Shots	Method	ImageNet	SUN	Aircraft	EuroSAT	Cars	Food	Pets	Flowers	Caltech	DTD	UCF	Average
0	CLIP (ICML '21)	66.7	62.6	24.7	47.5	65.3	86.1	89.1	71.4	92.9	43.6	66.7	65.1
1	CoOp (4) (IJCV '22)	68.0	67.3	26.2	50.9	67.1	82.6	90.3	72.7	93.2	50.1	70.7	67.2
	CoOp (16) (IJCV '22)	65.7	67.0	20.8	56.4	67.5	84.3	90.2	78.3	92.5	50.1	71.2	67.6
	CoCoOp (CVPR '22)	69.4	68.7	28.1	55.4	67.6	84.9	91.9	73.4	94.1	52.6	70.4	68.8
	TIP-Adapter-F (ECCV '22)	69.4	67.2	28.8	<u>67.8</u>	67.1	85.8	90.6	83.8	94.0	51.6	73.4	<u>70.9</u>
	CLIP-Adapter (IJCV '23)	67.9	65.4	25.2	49.3	65.7	86.1	89.0	71.3	<u>92.0</u>	44.2	66.9	<u>65.7</u>
	PLOT++ (ICLR '23)	66.5	66.8	28.6	65.4	<u>68.8</u>	<u>86.2</u>	91.9	80.5	94.3	54.6	<u>74.3</u>	70.7
	KgCoOp (CVPR '23)	68.9	68.4	26.8	61.9	66.7	86.4	<u>92.1</u>	74.7	<u>94.2</u>	52.7	72.8	69.6
	TaskRes (CVPR '23)	69.6	68.1	31.3	65.4	68.8	84.6	90.2	81.7	93.6	53.8	71.7	70.8
	MaPLe (CVPR '23)	<u>69.7</u>	<u>69.3</u>	28.1	29.1	67.6	85.4	91.4	74.9	93.6	50.0	71.1	66.4
	ProGrad (ICCV '23)	67.0	67.0	28.8	57.0	68.2	84.9	91.4	80.9	93.5	52.8	73.3	69.5
	CLIP-LoRA (Ours)	70.4	70.4	<u>30.2</u>	72.3	70.1	84.3	92.3	<u>83.2</u>	93.7	<u>54.3</u>	76.3	72.5
2	CoOp (4) (IJCV '22)	68.7	68.0	28.1	66.2	70.5	82.6	89.9	80.9	93.0	53.7	73.5	70.5
	CoOp (16) (IJCV '22)	67.0	67.0	25.9	65.1	70.4	84.4	89.9	88.0	93.1	54.1	74.1	70.8
	CoCoOp (CVPR '22)	70.1	69.4	29.3	61.8	68.4	85.9	<u>91.9</u>	77.8	94.4	52.3	73.4	70.4
	TIP-Adapter-F (ECCV '22)	70.0	68.6	<u>32.8</u>	73.2	70.8	86.0	91.6	90.1	93.9	<u>57.8</u>	76.2	73.7
	CLIP-Adapter (IJCV '23)	68.2	67.2	27.0	51.2	66.6	86.2	89.7	71.7	93.4	45.4	68.4	66.8
	PLOT++ (ICLR '23)	68.3	68.1	31.1	<u>76.8</u>	73.2	86.3	92.3	<u>89.8</u>	<u>94.7</u>	56.7	<u>76.8</u>	<u>74.0</u>
	KgCoOp (CVPR '23)	69.6	69.6	28.0	69.2	68.2	86.6	92.3	79.8	94.5	55.3	74.6	71.6
	TaskRes (CVPR '23)	<u>70.2</u>	70.5	32.7	70.2	72.1	85.6	90.7	84.4	94.3	55.6	75.2	72.9
	MaPLe (CVPR '23)	70.0	<u>70.7</u>	29.5	59.4	68.5	<u>86.5</u>	91.8	79.8	94.9	50.6	74.0	70.5
	ProGrad (ICCV '23)	69.1	69.0	31.1	66.3	72.4	84.8	91.5	87.5	93.6	56.0	75.6	72.4
	CLIP-LoRA (Ours)	70.8	71.3	33.2	82.7	73.2	83.2	91.3	<u>89.8</u>	94.6	59.9	80.0	75.5
4	CoOp (4) (IJCV '22)	69.7	70.6	29.7	65.8	73.4	83.5	92.3	86.6	94.5	58.5	78.1	73.0
	CoOp (16) (IJCV '22)	68.8	69.7	30.9	69.7	74.4	84.5	92.5	92.2	94.5	59.5	77.6	74.0
	CoCoOp (CVPR '22)	70.6	70.4	30.6	61.7	69.5	86.3	<u>92.7</u>	81.5	94.8	55.7	75.3	71.7
	TIP-Adapter-F (ECCV '22)	70.7	70.8	<u>35.7</u>	76.8	74.1	86.5	91.9	92.1	94.8	59.8	78.1	75.6
	CLIP-Adapter (IJCV '23)	68.6	68.0	27.9	51.2	67.5	86.5	90.8	73.1	94.0	46.1	70.6	67.7
	PLOT++ (ICLR '23)	70.4	71.7	35.3	<u>83.2</u>	<u>76.3</u>	86.5	92.6	<u>92.9</u>	<u>95.1</u>	<u>62.4</u>	<u>79.8</u>	<u>76.9</u>
	KgCoOp (CVPR '23)	69.9	71.5	32.2	71.8	69.5	86.9	92.6	87.0	95.0	58.7	77.6	73.9
	TaskRes (CVPR '23)	<u>71.0</u>	<u>72.7</u>	33.4	74.2	76.0	86.0	91.9	85.0	95.0	60.1	76.2	74.7
	MaPLe (CVPR '23)	70.6	71.4	30.1	69.9	70.1	<u>86.7</u>	93.3	84.9	95.0	59.0	77.1	73.5
	ProGrad (ICCV '23)	70.2	71.7	34.1	69.6	75.0	85.4	92.1	91.1	94.4	59.7	77.9	74.7
	CLIP-LoRA (Ours)	71.4	72.8	37.9	84.9	77.4	82.7	91.0	93.7	95.2	63.8	81.1	77.4
8	CoOp (4) (IJCV '22)	70.8	72.4	37.0	74.7	76.8	83.3	92.1	95.0	94.7	63.7	79.8	76.4
	CoOp (16) (IJCV '22)	70.6	71.9	38.5	77.1	79.0	82.7	91.3	94.9	94.5	64.8	80.0	76.8
	CoCoOp (CVPR '22)	70.8	71.5	32.4	69.1	70.4	<u>87.0</u>	93.3	86.3	94.9	60.1	75.9	73.8
	TIP-Adapter-F (ECCV '22)	<u>71.7</u>	73.5	39.5	81.3	78.3	86.9	91.8	94.3	95.2	<u>66.7</u>	82.0	78.3
	CLIP-Adapter (IJCV '23)	69.1	71.7	30.5	61.6	70.7	86.9	91.9	83.3	94.5	50.5	76.2	71.5
	PLOT++ (ICLR '23)	71.3	73.9	<u>41.4</u>	<u>88.4</u>	<u>81.3</u>	86.6	93.0	95.4	<u>95.5</u>	66.5	<u>82.8</u>	<u>79.6</u>
	KgCoOp (CVPR '23)	70.2	72.6	34.8	73.9	72.8	<u>87.0</u>	93.0	91.5	95.1	65.6	80.0	76.0
	TaskRes (CVPR '23)	72.3	<u>74.6</u>	40.3	77.5	79.6	86.4	92.0	<u>96.0</u>	95.3	<u>66.7</u>	81.6	78.4
	MaPLe (CVPR '23)	71.3	73.2	33.8	82.8	71.3	87.2	<u>93.1</u>	90.5	95.1	63.0	79.5	76.4
	ProGrad (ICCV '23)	71.3	73.0	37.7	77.8	78.7	86.1	92.2	95.0	94.8	63.9	80.5	77.4
	CLIP-LoRA (Ours)	72.3	74.7	45.7	89.7	82.1	83.1	91.7	96.3	95.6	67.5	84.1	80.3
16	CoOp (4) (IJCV '22)	71.5	74.6	40.1	83.5	79.1	85.1	92.4	96.4	95.5	69.2	81.9	79.0
	CoOp (16) (IJCV '22)	71.9	74.9	43.2	85.0	82.9	84.2	92.0	96.8	95.8	69.7	83.1	80.0
	CoCoOp (CVPR '22)	71.1	72.6	33.3	73.6	72.3	87.4	<u>93.4</u>	89.1	95.1	63.7	77.2	75.4
	TIP-Adapter-F (ECCV '22)	<u>73.4</u>	<u>76.0</u>	44.6	85.9	82.3	86.8	92.6	96.2	95.7	70.8	83.9	80.7
	CLIP-Adapter (IJCV '23)	69.8	<u>74.2</u>	34.2	71.4	74.0	87.1	92.3	92.9	94.9	59.4	80.2	75.5
	PLOT++ (ICLR '23)	72.6	<u>76.0</u>	<u>46.7</u>	<u>92.0</u>	<u>84.6</u>	87.1	93.6	<u>97.6</u>	<u>96.0</u>	71.4	<u>85.3</u>	<u>82.1</u>
	KgCoOp (CVPR '23)	70.4	73.3	36.5	76.2	74.8	<u>87.2</u>	93.2	93.4	95.2	68.7	81.7	77.3
	TaskRes (CVPR '23)	73.0	76.1	44.9	82.7	83.5	86.9	92.4	97.5	95.8	71.5	84.0	80.8
	MaPLe (CVPR '23)	71.9	74.5	36.8	87.5	74.3	87.4	93.2	94.2	95.4	68.4	81.4	78.6
	ProGrad (ICCV '23)	72.1	75.1	43.0	83.6	82.9	85.8	92.8	96.6	95.9	68.8	82.7	79.9
	CLIP-LoRA (Ours)	73.6	76.1	54.7	92.1	86.3	84.2	92.4	98.0	96.4	72.0	86.7	83.0

Table 4. Detailed results for the 11 datasets with ViT-B/32 as backbone. Top-1 accuracy averaged over 3 random seeds is reported. Highest value is highlighted in **bold**, and the second highest is underlined.

Shots	Method	ImageNet	SUN	Aircraft	EuroSAT	Cars	Food	Pets	Flowers	Caltech	DTD	UCF	Average
0	CLIP (ICML '21)	61.9	62.0	19.3	45.1	60.4	80.5	87.5	67.0	91.1	42.6	62.2	61.8
1	CoOp (4) (IJCV '22)	62.7	64.8	18.7	49.3	60.5	74.4	87.7	68.0	91.5	47.1	66.5	62.8
	CoOp (16) (IJCV '22)	60.8	63.3	15.7	52.9	59.6	75.9	87.6	71.5	91.7	47.2	66.3	63.0
	CoCoOp (CVPR '22)	64.3	65.6	15.5	46.7	60.4	79.5	88.2	68.6	92.2	45.7	67.3	63.1
	TIP-Adapter-F (ECCV '22)	64.5	65.4	<u>22.2</u>	59.7	61.1	80.4	87.5	81.1	<u>92.4</u>	<u>50.9</u>	66.5	<u>66.5</u>
	CLIP-Adapter (IJCV '23)	63.2	63.2	19.8	46.9	60.7	<u>80.6</u>	87.6	66.7	91.6	44.1	62.9	<u>62.5</u>
	PLOT++ (ICLR '23)	60.9	64.5	22.0	67.3	<u>61.4</u>	79.2	89.4	73.7	93.0	50.6	<u>69.7</u>	<u>66.5</u>
	KgCoOp (CVPR '23)	63.9	65.7	20.9	55.4	61.2	80.7	88.6	68.1	92.2	50.2	66.8	64.9
	TaskRes (CVPR '23)	<u>64.6</u>	62.0	20.9	60.1	59.6	74.6	84.3	75.2	88.8	50.1	64.3	64.0
	MaPLe (CVPR '23)	<u>64.6</u>	<u>66.9</u>	13.7	32.3	60.1	79.7	87.6	68.7	92.0	44.5	66.4	61.5
	ProGrad (ICCV '23)	62.0	64.9	21.2	52.8	60.6	78.1	87.9	74.9	91.6	51.0	66.5	64.7
	CLIP-LoRA (Ours)	65.2	67.6	22.9	<u>67.1</u>	63.5	78.4	<u>88.7</u>	<u>77.4</u>	93.0	49.8	72.3	67.8
2	CoOp (4) (IJCV '22)	63.3	65.6	20.8	56.5	62.1	74.2	86.6	77.1	91.5	49.4	71.5	65.3
	CoOp (16) (IJCV '22)	61.3	63.8	18.6	62.4	62.3	73.3	85.8	82.0	91.2	49.6	70.4	65.5
	CoCoOp (CVPR '22)	64.8	66.8	15.4	51.7	61.1	80.8	88.7	70.0	92.7	50.3	70.0	64.8
	TIP-Adapter-F (ECCV '22)	64.9	66.8	24.6	67.4	63.7	80.8	88.2	85.5	92.8	<u>54.9</u>	71.8	<u>69.2</u>
	CLIP-Adapter (IJCV '23)	63.4	64.8	20.9	50.1	61.5	<u>81.0</u>	87.9	67.1	91.4	44.9	65.9	63.5
	PLOT++ (ICLR '23)	62.6	65.5	23.1	<u>73.0</u>	<u>64.7</u>	77.0	87.4	84.3	92.9	52.8	<u>72.6</u>	68.7
	KgCoOp (CVPR '23)	64.2	67.5	21.7	58.5	62.4	81.1	88.8	73.7	92.8	53.1	69.2	66.6
	TaskRes (CVPR '23)	<u>65.2</u>	64.4	22.4	64.8	62.8	75.9	85.2	78.0	89.9	53.6	67.8	66.4
	MaPLe (CVPR '23)	<u>65.2</u>	<u>67.6</u>	20.5	46.9	61.9	80.7	89.3	71.6	<u>93.0</u>	49.7	71.0	65.2
	ProGrad (ICCV '23)	63.7	67.5	22.3	56.7	64.0	78.3	87.0	82.2	92.4	52.1	71.4	67.1
	CLIP-LoRA (Ours)	65.7	68.6	<u>24.1</u>	80.8	64.8	76.3	86.6	<u>84.7</u>	93.7	57.4	75.4	70.7
4	CoOp (4) (IJCV '22)	64.8	68.2	22.1	62.1	65.2	76.5	<u>90.1</u>	84.1	93.1	55.3	73.5	68.6
	CoOp (16) (IJCV '22)	63.2	67.1	24.0	68.7	66.2	75.6	88.8	87.9	93.0	55.3	75.0	69.5
	CoCoOp (CVPR '22)	65.2	67.8	17.3	58.5	62.0	81.1	89.8	74.6	93.2	52.3	71.6	66.7
	TIP-Adapter-F (ECCV '22)	<u>65.8</u>	68.3	28.8	71.5	67.6	80.9	88.6	88.9	94.6	<u>58.0</u>	75.1	71.6
	CLIP-Adapter (IJCV '23)	63.7	65.6	21.3	49.9	62.2	<u>81.3</u>	88.4	68.3	92.0	47.2	67.3	64.3
	PLOT++ (ICLR '23)	64.6	69.2	26.2	<u>81.6</u>	68.5	77.8	89.1	90.2	93.9	57.2	<u>75.6</u>	<u>72.2</u>
	KgCoOp (CVPR '23)	64.7	69.2	22.6	64.9	63.2	81.2	89.5	76.8	93.8	55.1	71.6	68.4
	TaskRes (CVPR '23)	66.1	66.7	23.1	70.7	66.7	76.7	86.7	79.0	90.6	57.0	68.2	68.3
	MaPLe (CVPR '23)	65.6	69.4	23.4	64.7	62.2	81.4	90.5	78.1	94.0	55.0	70.9	68.7
	ProGrad (ICCV '23)	65.2	<u>69.6</u>	24.8	63.7	66.4	79.2	89.4	87.5	93.2	55.9	73.4	69.8
	CLIP-LoRA (Ours)	66.5	70.3	<u>27.7</u>	85.6	<u>68.3</u>	75.6	86.3	<u>90.1</u>	<u>94.3</u>	60.3	76.5	72.9
8	CoOp (4) (IJCV '22)	65.8	70.0	28.0	72.8	69.0	77.2	89.5	90.2	93.8	60.8	77.1	72.2
	CoOp (16) (IJCV '22)	65.5	69.2	29.1	76.4	71.3	76.3	87.4	92.7	93.8	61.7	76.5	72.7
	CoCoOp (CVPR '22)	65.8	68.9	20.3	58.1	63.4	81.6	90.1	77.3	93.8	57.4	72.4	68.1
	TIP-Adapter-F (ECCV '22)	66.8	71.2	<u>32.1</u>	75.0	72.6	81.3	89.8	90.4	<u>94.5</u>	63.6	78.0	74.1
	CLIP-Adapter (IJCV '23)	64.2	69.3	23.5	55.2	65.4	81.5	89.3	78.0	93.9	50.8	73.0	67.6
	PLOT++ (ICLR '23)	66.2	71.0	31.7	<u>87.1</u>	73.5	78.2	88.4	93.8	94.4	62.9	<u>79.1</u>	<u>75.1</u>
	KgCoOp (CVPR '23)	65.1	69.5	24.7	66.2	65.0	<u>81.7</u>	<u>90.3</u>	83.1	<u>94.5</u>	61.1	74.7	70.5
	TaskRes (CVPR '23)	67.4	<u>71.9</u>	31.9	74.9	<u>73.8</u>	80.6	89.1	<u>93.5</u>	94.8	64.5	78.4	74.6
	MaPLe (CVPR '23)	66.3	70.3	25.4	79.0	63.7	81.9	90.9	81.1	94.4	59.8	75.0	71.6
	ProGrad (ICCV '23)	66.1	71.1	29.0	73.5	71.8	80.0	89.1	92.1	94.2	62.3	75.7	73.2
	CLIP-LoRA (Ours)	<u>67.2</u>	72.1	36.1	88.8	74.4	76.7	87.7	92.4	94.8	<u>63.7</u>	80.1	75.8
16	CoOp (4) (IJCV '22)	66.7	72.1	31.1	80.6	71.7	79.8	89.4	93.1	95.0	64.3	78.2	74.7
	CoOp (16) (IJCV '22)	66.8	72.2	32.9	83.3	76.0	78.6	88.7	95.4	94.9	65.3	78.6	75.7
	CoCoOp (CVPR '22)	66.0	69.8	22.6	70.4	64.6	<u>81.9</u>	<u>91.0</u>	82.5	94.3	59.7	75.3	70.7
	TIP-Adapter-F (ECCV '22)	68.4	74.1	34.8	83.4	77.0	81.7	90.4	94.3	95.1	68.0	80.5	77.1
	CLIP-Adapter (IJCV '23)	64.9	71.8	26.7	64.7	68.9	<u>81.9</u>	90.1	88.7	94.8	58.1	76.5	71.6
	PLOT++ (ICLR '23)	67.4	73.4	36.3	<u>91.1</u>	77.4	79.7	89.1	96.3	94.9	67.0	<u>81.5</u>	<u>77.6</u>
	KgCoOp (CVPR '23)	65.4	71.0	23.7	70.1	67.3	81.7	90.8	86.1	94.4	65.1	77.5	72.1
	TaskRes (CVPR '23)	<u>68.2</u>	73.6	<u>37.0</u>	77.7	<u>78.0</u>	81.4	89.4	95.5	95.7	68.3	80.6	76.9
	MaPLe (CVPR '23)	66.7	72.0	28.0	83.3	66.9	82.1	91.7	89.0	95.1	63.4	77.3	74.1
	ProGrad (ICCV '23)	66.9	73.2	33.3	81.0	76.1	80.1	89.3	95.1	95.0	65.8	79.6	75.9
	CLIP-LoRA (Ours)	68.4	<u>74.0</u>	44.9	91.8	79.7	78.2	88.8	<u>96.2</u>	<u>95.2</u>	<u>68.2</u>	82.8	78.9

Table 5. Detailed results for the 11 datasets with ViT-L/14 as backbone. Top-1 accuracy averaged over 3 random seeds is reported. Highest value is highlighted in **bold**, and the second highest is underlined.

Shots	Method	ImageNet	SUN	Aircraft	EuroSAT	Cars	Food	Pets	Flowers	Caltech	DTD	UCF	Average
0	CLIP (ICML '21)	72.9	67.6	32.6	58.0	76.8	91.0	93.6	79.4	94.9	53.6	74.2	72.2
1	CoOp (4) (IJCV '22)	73.9	70.9	35.8	64.3	78.8	89.3	93.0	86.5	93.8	58.0	77.4	74.7
	CoOp (16) (IJCV '22)	71.5	68.9	36.9	68.3	78.6	89.0	94.0	87.2	94.9	58.5	78.5	75.1
	CoCoOp (CVPR '22)	76.0	72.1	36.5	65.0	78.8	<u>90.8</u>	94.7	83.0	95.7	57.5	78.5	75.3
	TIP-Adapter-F (ECCV '22)	76.4	71.0	38.5	67.8	79.2	91.0	93.2	<u>90.9</u>	95.3	59.3	77.9	76.4
	CLIP-Adapter (IJCV '23)	74.6	69.3	32.9	60.1	77.2	91.0	93.6	79.5	94.9	53.7	74.7	72.9
	PLOT++ (ICLR '23)	73.7	71.1	35.2	72.4	78.9	89.4	93.6	85.0	<u>95.6</u>	58.9	<u>80.7</u>	75.9
	KgCoOp (CVPR '23)	75.5	72.2	35.9	69.5	77.9	91.0	94.1	82.2	95.5	61.2	77.7	75.7
	TaskRes (CVPR '23)	76.2	71.4	<u>39.7</u>	70.6	<u>79.8</u>	89.9	93.5	87.6	94.6	59.8	77.4	76.4
	MaPLe (CVPR '23)	<u>76.5</u>	<u>73.3</u>	37.4	61.2	79.1	90.2	94.4	83.6	95.2	58.4	78.5	75.3
	ProGrad (ICCV '23)	73.6	71.1	38.3	70.8	<u>79.8</u>	90.5	<u>94.5</u>	88.8	95.5	58.5	80.1	76.5
	CLIP-LoRA (Ours)	76.7	74.3	41.2	73.7	80.7	90.5	94.2	91.2	95.7	61.2	82.4	78.3
2	CoOp (4) (IJCV '22)	74.6	70.7	38.4	71.4	81.0	89.5	93.7	92.7	95.6	60.8	81.1	77.2
	CoOp (16) (IJCV '22)	73.1	69.3	38.8	72.7	81.2	89.4	93.5	93.5	95.0	59.2	80.0	76.9
	CoCoOp (CVPR '22)	76.3	73.0	38.9	72.5	79.7	<u>91.2</u>	94.4	87.1	96.2	60.7	80.8	77.3
	TIP-Adapter-F (ECCV '22)	76.6	72.6	<u>42.2</u>	76.7	80.4	91.1	93.8	<u>93.8</u>	95.5	60.3	80.4	<u>78.5</u>
	CLIP-Adapter (IJCV '23)	74.9	71.0	34.0	61.9	78.1	<u>91.2</u>	93.6	79.8	95.2	55.6	76.1	73.8
	PLOT++ (ICLR '23)	75.0	71.4	40.6	<u>76.8</u>	80.7	89.6	93.5	93.6	<u>96.7</u>	<u>65.0</u>	80.3	<u>78.5</u>
	KgCoOp (CVPR '23)	76.0	73.9	38.1	76.5	79.3	91.4	<u>94.3</u>	83.6	96.1	63.3	78.9	77.4
	TaskRes (CVPR '23)	76.6	73.1	42.0	74.4	<u>81.7</u>	90.6	93.8	89.8	95.7	62.5	79.2	78.1
	MaPLe (CVPR '23)	<u>76.9</u>	<u>74.9</u>	38.3	70.9	79.2	<u>91.2</u>	94.2	87.7	96.4	61.3	<u>81.3</u>	77.5
	ProGrad (ICCV '23)	75.1	72.6	41.1	73.7	82.3	90.6	94.1	<u>93.8</u>	96.1	61.5	81.1	78.4
	CLIP-LoRA (Ours)	77.3	75.5	42.4	85.9	82.3	89.8	93.2	94.7	96.8	67.3	84.2	80.9
4	CoOp (4) (IJCV '22)	76.0	73.7	41.8	77.9	82.6	88.8	94.7	94.9	96.1	64.3	83.6	79.5
	CoOp (16) (IJCV '22)	74.9	73.1	43.6	75.9	83.3	88.7	94.6	<u>95.9</u>	96.5	63.9	82.8	79.4
	CoCoOp (CVPR '22)	77.0	74.7	41.0	74.7	79.7	<u>91.3</u>	<u>94.9</u>	89.8	<u>97.1</u>	64.9	82.6	78.9
	TIP-Adapter-F (ECCV '22)	77.1	74.1	<u>47.4</u>	<u>81.4</u>	82.3	<u>91.2</u>	94.0	95.5	96.5	64.4	<u>83.9</u>	<u>80.7</u>
	CLIP-Adapter (IJCV '23)	75.2	72.1	35.8	61.3	78.8	91.2	93.7	81.7	95.6	57.9	77.9	74.7
	PLOT++ (ICLR '23)	76.4	75.2	43.2	81.3	82.6	87.7	94.2	<u>95.9</u>	96.9	<u>66.8</u>	83.8	80.4
	KgCoOp (CVPR '23)	76.4	75.2	40.6	79.5	80.0	91.5	94.4	90.2	96.9	66.3	83.4	79.5
	TaskRes (CVPR '23)	77.1	74.9	42.5	76.6	83.6	90.7	94.4	90.3	96.5	65.4	80.1	79.3
	MaPLe (CVPR '23)	<u>77.2</u>	<u>76.0</u>	40.4	74.6	80.3	91.5	95.0	93.2	97.0	64.5	82.8	79.3
	ProGrad (ICCV '23)	76.5	75.0	44.6	79.3	<u>83.8</u>	90.6	94.8	95.6	96.8	66.3	83.6	80.6
	CLIP-LoRA (Ours)	77.9	76.7	48.9	86.4	85.2	89.6	93.9	97.4	97.2	70.4	86.4	82.7
8	CoOp (4) (IJCV '22)	77.2	75.5	48.7	81.4	85.9	89.2	94.5	97.5	96.5	68.8	86.0	81.9
	CoOp (16) (IJCV '22)	76.8	75.0	<u>51.2</u>	82.8	<u>86.4</u>	88.6	94.0	98.0	96.7	69.4	85.1	82.2
	CoCoOp (CVPR '22)	77.4	75.6	43.3	77.0	81.4	91.6	95.3	93.0	<u>97.0</u>	67.9	84.5	80.4
	TIP-Adapter-F (ECCV '22)	77.8	76.7	50.4	84.9	85.9	<u>91.4</u>	94.1	97.3	96.9	<u>71.2</u>	<u>86.2</u>	<u>83.0</u>
	CLIP-Adapter (IJCV '23)	75.7	75.9	40.7	67.9	81.6	<u>91.4</u>	94.3	92.3	96.8	63.8	82.8	78.5
	PLOT++ (ICLR '23)	77.8	77.0	43.2	<u>87.0</u>	84.6	89.6	93.3	96.3	96.8	69.5	84.8	81.8
	KgCoOp (CVPR '23)	76.7	76.2	45.9	82.1	82.3	91.6	<u>95.1</u>	95.2	97.3	70.8	85.7	81.7
	TaskRes (CVPR '23)	77.9	76.0	51.1	81.1	85.7	91.1	94.5	96.7	96.9	69.4	85.6	82.4
	MaPLe (CVPR '23)	<u>78.0</u>	<u>77.2</u>	42.9	80.7	81.8	90.1	95.0	95.8	96.8	69.5	85.1	81.2
	ProGrad (ICCV '23)	77.7	76.1	49.9	83.6	86.2	90.8	<u>95.1</u>	<u>97.8</u>	96.7	69.9	85.4	82.7
	CLIP-LoRA (Ours)	78.5	78.0	57.5	90.0	88.7	89.7	94.2	98.0	<u>97.0</u>	72.2	88.3	84.7
16	CoOp (4) (IJCV '22)	78.1	77.9	53.0	86.7	87.4	90.2	94.5	98.6	97.5	73.7	86.7	84.0
	CoOp (16) (IJCV '22)	78.2	77.5	55.2	88.3	<u>89.0</u>	89.8	94.6	99.1	97.2	74.4	87.3	84.6
	CoCoOp (CVPR '22)	77.8	76.7	45.2	79.8	<u>82.7</u>	<u>91.9</u>	95.4	95.3	<u>97.4</u>	71.4	85.2	81.7
	TIP-Adapter-F (ECCV '22)	<u>79.3</u>	79.6	<u>55.8</u>	86.1	88.1	91.6	94.6	98.3	97.5	74.0	87.4	84.8
	CLIP-Adapter (IJCV '23)	76.4	78.0	<u>46.4</u>	75.8	83.8	91.6	94.3	97.3	97.3	71.3	86.1	81.7
	PLOT++ (ICLR '23)	78.6	79.1	44.1	<u>92.2</u>	87.2	90.2	93.6	98.8	97.5	<u>75.0</u>	87.1	83.9
	KgCoOp (CVPR '23)	76.8	76.7	47.5	<u>83.6</u>	83.2	91.7	<u>95.3</u>	96.4	<u>97.4</u>	73.6	86.4	82.6
	TaskRes (CVPR '23)	78.1	76.9	55.0	84.3	87.6	91.5	94.7	97.8	97.3	74.4	86.6	84.0
	MaPLe (CVPR '23)	78.4	78.8	46.3	85.4	83.6	92.0	95.4	97.4	97.2	72.7	86.5	83.1
	ProGrad (ICCV '23)	78.4	78.3	55.6	89.3	88.8	90.8	94.9	98.7	97.5	73.7	<u>87.7</u>	84.9
	CLIP-LoRA (Ours)	79.6	<u>79.4</u>	66.2	93.1	90.9	89.9	94.3	<u>99.0</u>	97.3	76.5	89.9	86.9